

Assessment Brief

CA1 – Deploying to Production

Module Name	Lecturer Name
Web Development and Delivery	Michael McAndrew

Title of Brief
CA1 – Deploying to Production

Percentage of Overall Grade	50%
Date handed out	Wednesday 30th September
Due date	Sunday 8th November
Individual or Group	Individual

Problem to solve

Dropbox is a cloud-based file hosting and storage service that allows users to save, sync, and share files on the web. You have been hired as an early engineer at Dropbox. The app runs on developers' local environments but hasn't been deployed yet. The company wants to release it to a small number of users as a Beta.

Your task is to get the Dropbox file service running in production. It should be accessible at a domain name and secured with HTTPS. Beta users will report bugs and issues with the product, and the company has hired two developers who will be fixing bugs and making product changes in response to this feedback. You will be required to make releasing changes to the product an easy and fast process for these developers.

You will be provided with an existing codebase for the Dropbox application.

Requirements

Technical Stack

You must use:

- **Docker** to containerise the application
- **AWS** compute infrastructure to deploy the application
- **GitHub** to manage the codebase for the application

You are free to use any tools you wish for other purposes, provided that you justify your choices.

The Service

- Both the frontend and the backend API should be deployed, and the application should be reachable at a public domain
- It should be served over HTTPS with a valid TLS certificate
- Application data should be protected at rest and in transit

Configuration and Secrets

- The application configuration is read from the environment and no environment-specific values should be stored in the codebase.
- Best practices are followed for securely storing and managing application secrets.

Local Development

- A developer who has never worked in your repository before can clone it, and set it up in their development environment by following instructions that you provide in the repo.

Delivery

- A change merged to your main branch, including any database migrations, should be deployed to production automatically, with no manual operations required by the developer
- Every pull request runs a set of automated checks to ensure that the code is safe to deploy
- Changes should only be merged to the main branch through pull requests, and a pull request cannot be merged unless its checks pass
- You can roll back the production deployed version of the application to a previous version quickly to respond to an incident, and instructions for doing so for a developer are easy to follow

Contributing

- Your repository has a CONTRIBUTING.md file with instructions for a new developer on how to set up a development environment, run checks locally, open a pull request, and development practices for the repo.
- **A pull request will be opened against your repository after you have submitted your work.** It will contain proposed changes for a new feature. You should review the pull request, read the code, ask questions, and request changes where needed. When merged, it should be deployed to production.

Extra Mile

This is your opportunity to take your deployment beyond the requirements, and what was covered in class. You can use the following for inspiration, or propose your own:

- Define the infrastructure for the service as code with OpenTofu or Terraform
- Create a staging environment with its own database, storage, and compute infrastructure, deployed by the same process
- Detect a failed release automatically and roll production back to a stable release without human intervention
- Create a temporary environment for every pull request, deployed when the pull request is created and destroyed when it is closed or merged
- A new application feature that requires infrastructure changes to work e.g. a bulk-export asynchronous task that creates a compressed archive of all of a user's files, using a task queue such as AWS SQS and a separate worker process, and gives the user a time-limited download link when it is ready

Presentation and Documentation

Architecture Document

Write an ARCHITECTURE.md of roughly 1,500 to 2,000 words. You should write this document as if it is documentation for a colleague who wishes to understand how the system works. You should include:

- **Overview:** what the service is and what it runs on
- **Architecture diagram:** A diagram of your deployment showing every component and how they connect together
- **Infrastructure:** describe each piece of infrastructure that you use, what it does, and how it is configured. Include the actual security group rules and IAM policies you applied. This should cover all roles and policies in your system, including:

- roles used by your compute (e.g. instance or task roles)
 - roles used by your deployment pipeline
 - resource policies, such as S3 bucket policies
 - any IAM users you created
- **Data and file storage:** Where application data and uploaded files are stored
 - **Configuration and secrets:** how the application gets its configuration, how it reads secrets in production, and how to add a new secret or configuration if it was needed
 - **Deployment pipeline:** how deployments are made, and the process involved from committing a code change to it being deployed on infrastructure
 - **Decisions and trade-offs:** What design, infrastructure, and tooling decisions you made, and why. Document any current shortcomings and proposals for future work to improve the system

Security group rules and IAM policies can be placed in an appendix at the end of the document, or linked to files in your repository. They do not count towards the word count.

Video Demonstration

Record a 15 minute video explaining what you have built. Include:

- A demo of the application running at the production URL
- The compute infrastructure in the AWS console including security and configuration settings
- A demo of a simple code change working through the deployment pipeline and an explanation of how the pipeline works

Q&A

You will be required to attend a short, one-to-one Q&A session during class time after the submission deadline. This session is a test of your understanding of the system that you built. You will be asked to explain how parts of your system works, and justify the decisions you made to build it.

You must attend the Q&A to receive a grade for your submission.

Submission

The deadline for submission is **Sunday 8th November at 23:59**. For each day that your assignment is submitted late, 5% will be deducted. Assignments submitted more than 7 days late will not be accepted. Academic integrity guidance for this project is available on the module website.

- A link to a private GitHub repository with the lecturer (mcandru) added as a collaborator. Your commit history should show steady, incremental progress throughout
- A zip export of your repository at the point of submission
- A README.md with setup instructions and the live URL for your deployment
- The ARCHITECTURE.md document
- A CONTRIBUTING.md document with setup and contribution instructions
- All pipeline and configuration files committed to the repository
- The video demonstration screencast

Marking Scheme

Category	Description	Weighting
Cloud Deployment	Application is deployed to AWS and reachable at a public domain over HTTPS with a valid certificate. Application is containerised with Docker and the infrastructure is configured correctly. Application data and uploaded files are stored appropriately in production.	15%
System Design and Architecture	Architecture diagram and description accurately represent the deployed system and how its components connect. Infrastructure choices are appropriate for the problem, with clear reasoning for decisions and trade-offs. Excellent understanding of the AWS services used and how they fit together.	10%
Developer Experience	A new developer can clone the repository and set up a working local environment by following the provided instructions. Merging to main deploys to production automatically, including database migrations, and every pull request runs automated checks that must pass before merge. Production can be rolled back to a previous version quickly, with a documented process. The provided pull request is reviewed thoroughly, with useful comments and requested changes, then merged and deployed.	20%
Security and privacy	Secrets and environment-specific values are separate from the codebase and managed following best practices. Security group rules and IAM policies follow least privilege. Application data is protected at rest and in transit in production.	15%
Documentation	ARCHITECTURE.md explains the system clearly to a colleague who has not seen it before, covering infrastructure, data, configuration, and the deployment pipeline. README.md and CONTRIBUTING.md give a new developer the instructions needed to set up, run checks, and contribute. Writing is clear and well structured.	10%
Presentation and Q&A	Video clearly demonstrates the working application, the AWS infrastructure, and a change moving through the deployment pipeline. Clear, accurate explanation of how each part of the system works. Sound justification for the decisions and trade-offs made. Demonstrates genuine understanding and ownership of the work.	20%
Extra Mile	Goes meaningfully beyond the core requirements. The extension must change the application infrastructure or deployment pipeline. Changes to the application codebase alone are not sufficient. Technical depth and quality of the chosen extension. Extension is properly integrated with the rest of the system.	10%